DAT22-1

Scraping, Pandas II & Algorithmic Thinking

Collecting your own data, and processing it at scale

First get the data; then make it fast enough to matter.

Albert School — 28 hours

Preface

In Pandas I (DAT12-3) you learned to take a dataset someone handed you — a CSV, an Excel sheet — and bend it into shape: filter it, group it, merge it, reshape it, and write a paragraph saying what it means. In Programming in Python (DAT12-2) you learned the language those operations are written in, and you made your first HTTP request to an API. This course removes the two comforts you have relied on so far.

The first comfort was that **someone gave you the data**. Here you will go and get it yourself, from web pages that were never designed to be read by a program. The second comfort was that **the data was small enough that speed never mattered**. Once you collect your own data, the volume grows, and an operation that took a tenth of a second on a thousand rows can take ten minutes on a million. So this course braids together three strands that, in practice, are inseparable:

- **Advanced pandas** — the operations Pandas I had no room for: hierarchical indices, windowed calculations, and the special powers of a time axis.
- **Algorithmic thinking** — a vocabulary (Big O) and a habit (profiling) for answering the question "will this still work when the data is ten times bigger?" **before** it is too late to change course.
- **Web scraping** — turning a public web page into a clean DataFrame, and doing it legally, ethically, and reproducibly.

These are taught together on purpose. You meet Big O not as abstract theory but at the exact moment you are tempted to write a Python loop over a DataFrame and a vectorized one-liner would be a thousand times faster. You meet recursion not on toy puzzles but when a scraped JSON document nests boxes inside boxes and a flat loop simply cannot reach the bottom. The tagline says it plainly: **first get the data; then make it fast enough to matter.** By the end you will collect a dataset that did not exist before you built the collector, and you will know — not guess — whether your pipeline can survive being scaled up.

Contents

1 Advanced pandas

1.1 Beyond the flat table

Pandas I treated a DataFrame as a flat grid: one row index, one column index, each a simple list of labels. That model carries you a long way, but real data is often **hierarchical** (sales by country **and** by year) or **sequential** (a reading every minute, where "the previous row" has meaning). This chapter adds the three pandas features that handle exactly these shapes: multi-level indexing, window functions, and the time axis.

1.2 Multi-level indexing

Imagine total revenue broken down by country and, within each country, by year. In a flat table you would repeat the country name on every row. A **MultiIndex** (also called a hierarchical index) instead lets a single axis carry **several levels** of labels at once.

> **Worked example — A naturally two-level table.** Sales for two countries over two years:
>
> | Country | Year | Revenue (k€) |
> |---|---|---|
> | France | 2025 | 120 |
> | France | 2026 | 135 |
> | Spain | 2025 | 80 |
> | Spain | 2026 | 95 |
>
> The pair `(Country, Year)` identifies each row. Setting **both** columns as the index — `df.set_index(["Country", "Year"])` — turns `Country` into the outer level and `Year` into the inner level of a single MultiIndex.

Definition 1 (MultiIndex) A **MultiIndex** is an index whose labels are **tuples**, drawn from two or more ordered **levels**. Level 0 is the outermost. A row is addressed by giving a label for each level — `("France", 2026)` — or by giving only the outer level(s) to select a whole **block** of rows.

The figure shows how the levels nest. The outer label is written once and spans all the inner labels beneath it.

level 0 Country	level 1 Year	Revenue
France	2025	120
	2026	135
Spain	2025	80
	2026	95

Figure 1: A two-level MultiIndex. The outer label `France` spans both of its years; addressing `("France", 2026)` descends both levels, while `"France"` alone selects the whole France block.

To pull data out, `loc` now takes a tuple per level:

- `df.loc["France"]` — the whole France block (a smaller DataFrame, indexed by year).
- `df.loc[("France", 2026)]` — the single France/2026 row.

- `df.xs(2026, level="Year")` — a **cross-section**: every country's 2026 row, slicing across the inner level.

The complement of building a MultiIndex is collapsing one. `stack` moves a column level **down** into the row index (wide → long); `unstack` moves a row level **up** into the columns (long → wide). These are the hierarchical generalisation of the `pivot`/`melt` you met in Pandas I.

> **Worked example — unstack turns a level into columns.** Starting from the MultiIndexed table above, `df.unstack("Year")` lifts the `Year` level into the columns, giving one row per country and one column per year:
>
> | Country | 2025 | 2026 |
> |---|---|---|
> | France | 120 | 135 |
> | Spain | 80 | 95 |
>
> Same numbers, reshaped — now a year-on-year comparison reads straight across each row.

Common pitfall After a `groupby` on several keys, the result is MultiIndexed by those keys — this surprises people who then try `result["France"]` (column access) and get a `KeyError`. To grade an index level back into an ordinary column, use `reset_index()`. And once rows are MultiIndexed, `loc` expects a **tuple**, not two separate arguments: `df.loc[("France", 2026)]`, not `df.loc["France", 2026]` (the latter is read as row, **column**).

1.3 Window functions: rolling and expanding

A plain aggregation (`df["sales"].mean()`) collapses a whole column to one number. But many questions ask for a **running** summary — "the 7-day average sales as of each day", "the cumulative total so far". A **window function** computes an aggregate over a sliding or growing slice of the rows, returning one value **per row** rather than one value overall.

> **Worked example — Smoothing noisy daily sales.** Daily sales jump around: `10, 14, 9, 13, 30, 11, 12`. The single spike of `30` might be a one-off, not a trend. A **3-day rolling mean** replaces each day with the average of it and the two before it, smoothing the noise:
>
> ```
> df["sales"].rolling(window=3).mean()
> # NaN, NaN, 11.0, 12.0, 17.3, 18.0, 17.7
> ```
>
> The first two values are `NaN` because a 3-wide window cannot be filled until the third row. The lone spike is now spread across three days, so a genuine trend stands out from a single odd day.

Definition 2 (Rolling vs expanding windows) A **rolling window** of size k aggregates each row together with the $k-1$ rows before it — a **fixed-width** window that slides down the data. An **expanding window** aggregates each row together with **all** rows before it — a window that **grows** from the top, giving running (cumulative) statistics.

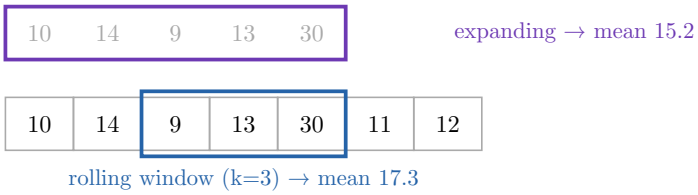

Figure 2: A rolling window (blue) keeps a fixed width as it slides; an expanding window (purple) starts at the top and grows by one row each step. Both emit one value per row.

Expanding statistics are written the same way: `df["sales"].expanding().mean()` gives the running average, `.expanding().sum()` the cumulative total. Both `rolling` and `expanding` accept any aggregate — `.mean()`, `.sum()`, `.std()`, `.max()`.

Common pitfall By default a rolling window is **trailing** (right-aligned): the value on row i uses rows $i-k+1..i$ and **never** peeks at the future. This matters enormously in forecasting: a **centered** window (`center=True`) uses rows on both sides, which is fine for smoothing a chart but is **data leakage** if the smoothed value will be used to predict the future — you would be feeding tomorrow's data into today's prediction.

1.4 Time-indexed DataFrames

When the index **is** time, pandas unlocks operations that make no sense on an arbitrary index. The key step is to parse your dates into real timestamps — `pd.to_datetime(df["date"])` — and set them as the index. A string column of dates is just text; a `DatetimeIndex` is a first-class time axis.

Definition 3 (DatetimeIndex and resampling) A **DatetimeIndex** is a row index whose labels are timestamps. With it you can: slice by time using natural strings (`df.loc["2026-01"]` selects all of January 2026); and **resample** — change the time granularity, e.g. `df.resample("M")` groups rows into calendar months. Resampling is a `groupby` whose key is a unit of time; like `groupby` it must be followed by an aggregate.

Worked example — Daily readings, monthly report. You have one temperature reading per day for a year (365 rows). To report the monthly average:

```
df = df.set_index(pd.to_datetime(df["date"]))
df["temp"].resample("M").mean()   # 12 values, one per month
```

`resample("M")` is to a time index what `groupby("month")` is to a category — but it understands that February is shorter, that months have an order, and that "2026-02" means a span of time, not a label.

Common pitfall The single most common time-series bug is leaving dates as **strings**. Sorted as text, `"2026-1-9"` comes after `"2026-1-10"` (because `'9' > '1'`), and `resample`/`.loc["2026-01"]` raise errors or return nonsense. Always `pd.to_datetime` first, and check `df.index.dtype` reads `datetime64`, not `object`.

Chapter takeaways. A MultiIndex carries several label levels on one axis; address it with tuples and reshape it with `stack`/`unstack`. Window functions emit one value per row over

a sliding (`rolling`) or growing (`expanding`) slice — use trailing windows to avoid leaking the future. A `DatetimeIndex` turns time into a first-class axis, enabling string-based slicing and `resample`.

2 Algorithmic complexity: Big O

2.1 The question "will this scale?"

Every operation in the previous chapter has a **cost** that grows with the size of the data. On a thousand rows you will not notice. On ten million you will wait, or crash. Big O notation is the vocabulary for talking about **how** that cost grows — not how many seconds something takes on your laptop, but how the time **scales** as the data gets bigger. It is the single most useful idea for deciding, before you press run, whether an approach is viable at all.

2.2 Counting operations, not seconds

> **Worked example — Two ways to check for duplicates.** You want to know whether a list of n order IDs contains any duplicate.
>
> **Approach A** — compare every pair: for each ID, loop over all the others looking for a match. With n IDs that is roughly $n \times n = n^2$ comparisons.
>
> **Approach B** — put the IDs into a Python `set` as you go; a set rejects duplicates and membership-checks are effectively instant. That is about n operations, one per ID.
>
> On $n = 1000$ IDs, Approach A does $1\,000\,000$ comparisons and B does $1\,000$ — a thousandfold gap. On $n = 1,!000,!000$ IDs, A does a **trillion** comparisons (minutes to hours) while B does a million (a blink). Same task, same answer, catastrophically different scaling.

Definition 4 (Big O notation) **Big O notation** describes how the number of operations an algorithm performs grows as a function of the input size n, keeping only the **fastest-growing term** and dropping constant factors. An algorithm is $O(f(n))$ if, for large n, its operation count grows no faster than a constant times $f(n)$.

Approach A above is $O(n^2)$ ("quadratic"); Approach B is $O(n)$ ("linear").

Why drop constants and lower terms? Because Big O answers a **scaling** question, not a stopwatch question. An algorithm doing $3n + 50$ operations and one doing n operations are both $O(n)$: double the data and both roughly double. But an $O(n^2)$ algorithm **quadruples** when the data doubles — and no constant factor saves it once n is large enough. The shape of the growth dominates everything else.

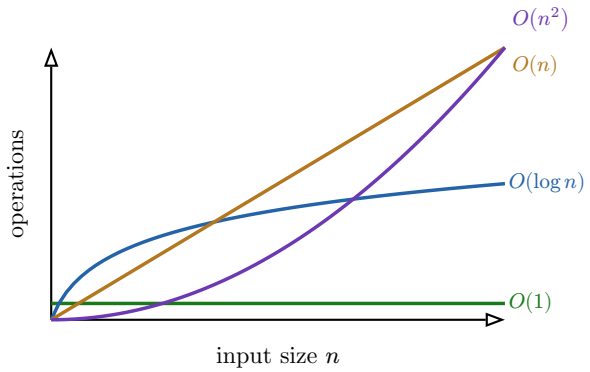

Figure 3: How operation count grows with input size. $O(1)$ stays flat and $O(\log n)$ grows slowly; $O(n)$ rises steadily; $O(n^2)$ curves sharply upward and overtakes everything else as n grows. Big O is about the **shape** of these curves, not their starting height.

Definition 5 (The complexity classes you will meet) In rough order of growth:

- $O(1)$ — **constant**: cost independent of n (looking up a dict key, `df.iloc[5]`).
- $O(\log n)$ — **logarithmic**: cost grows very slowly (binary search in sorted data).
- $O(n)$ — **linear**: cost proportional to n (one pass over a column).
- $O(n \log n)$ — typical of a good **sort**.
- $O(n^2)$ — **quadratic**: a loop inside a loop over the same data; the danger zone.

2.3 Why a loop over a DataFrame is the trap

The reason this chapter sits where it does: the most common $O(n^2)$ mistake in data work is writing a Python loop where pandas offers a vectorized operation.

Worked example — Nested loop vs vectorized join. You have a DataFrame of n orders and want to attach each customer's name from a customer table of size m. The beginner writes a loop: for each order, scan the customer table for a match — $n \times m$ operations, $O(n \times m)$. For large tables this is the quadratic trap in disguise.

The pandas way is `orders.merge(customers, on="customer_id")`. Internally pandas builds a hash index and matches in roughly $O(n + m)$ — and runs the inner work in optimised C rather than interpreted Python, a second large constant-factor win on top of the better complexity.

Common pitfall Big O hides the constant factor, and for pandas the constant factor is **huge**. A Python `for` loop over rows pays the interpreter's overhead on every iteration; a vectorized operation runs the same logic in compiled C over the whole array. So an $O(n)$ Python loop and an $O(n)$ vectorized call are **both** linear, yet the vectorized one is routinely 10–100× faster. Big O tells you which approaches are **viable**; the constant factor decides between two viable ones. You need both lenses.

Chapter takeaways. Big O describes how an algorithm's work grows with input size n, keeping only the dominant term. $O(n)$ scales gently; $O(n^2)$ — a loop within a loop over the same data — explodes. The classic data-work mistake is a Python loop over a DataFrame where a vectorized operation (`merge`, a boolean mask, a column arithmetic) is both lower-complexity and, thanks to compiled C, far lower constant factor.

3 Performance profiling

3.1 Measure before you optimise

Big O tells you which approaches **could** be slow; profiling tells you which ones **actually are** on your data. The cardinal rule of performance work is: never optimise by guesswork. Programmers are famously bad at guessing where time goes — the bottleneck is rarely where intuition points. **Measure first, optimise the measured bottleneck, measure again.**

3.2 Timing a piece of code

The quickest measurement is wall-clock time. In a notebook, the `%timeit` magic runs a snippet many times and reports a stable average; in a script, the `time.perf_counter()` clock brackets the code you care about.

> **Worked example — Timing two approaches head to head.**
>
> ```
> import time
> start = time.perf_counter()
> result = slow_loop(df)
> print(time.perf_counter() - start, "seconds")
> ```
>
> Run the same harness around the vectorized version and compare. A measurement like "loop: 4.1 s, vectorized: 0.02 s" turns the abstract claim of the previous chapter into a number you can put in a report — and a decision you can defend.

Definition 6 (Profiling and the bottleneck) **Profiling** is measuring where a program spends its time, function by function. The **bottleneck** is the part that dominates the total — often a single step responsible for most of the runtime. By **Amdahl's principle**, speeding up anything **other** than the bottleneck barely moves the total: halving a step that is 5% of the runtime saves 2.5%; halving the step that is 80% of the runtime saves 40%.

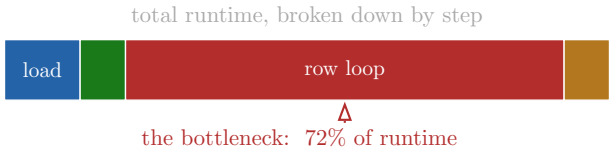

Figure 4: A profile of a pipeline. Optimising "load", "clean", or "save" can save at most a sliver; only attacking the row loop — the bottleneck — meaningfully cuts the total. Profile to find this bar before touching any code.

3.3 The optimise loop

Definition 7 (The profile–optimise cycle)

1. **Profile** the whole pipeline and find the step with the largest share of runtime.
2. **Optimise** that step — most often by replacing a Python loop with a vectorized pandas operation, or an $O(n^2)$ approach with an $O(n)$ one.
3. **Re-measure** to confirm a real gain **and** that the result is unchanged.
4. Repeat until fast enough — then **stop**.

> **Worked example — From four seconds to twenty milliseconds.** Profiling a pipeline shows 90% of the time in a loop that computes a discounted price row by row. Replacing it

with a vectorized column expression — `df["net"] = df["price"] * (1 - df["discount"])` — and re-measuring shows the total fall from 4 s to 0.02 s. The other steps were never worth touching; the profile said so before you wrote a line.

Common pitfall "Premature optimisation is the root of all evil" — optimising code that is already fast enough wastes your time and usually makes the code **harder to read**. A clear loop that runs once a day in two seconds needs no vectorizing. Optimise only what a measurement proves is too slow, and **stop** the moment it is fast enough. Readable-and-adequate beats clever-and-fragile.

Chapter takeaways. Measure, don't guess: time code with `%timeit` or `time.perf_counter()`, and profile to find the bottleneck — the step owning most of the runtime. Optimising anything else barely helps. Apply the profile–optimise–re-measure loop, usually by vectorizing the hot loop, and stop once it is fast enough.

4 Recursion

4.1 When a loop cannot reach the bottom

Some data is **nested to an unknown depth**: a JSON document where a comment has replies, which have replies, which have replies; an HTML page where a `<div>` contains `<div>`s containing `<div>`s. A flat `for` loop handles one level. To process **arbitrarily** deep nesting, you need a function that handles one level and then **calls itself** on what is inside — a **recursive** function. This course introduces recursion exactly here, because nested scraped data is where you will genuinely need it.

4.2 The shape of a recursive function

Worked example — Summing a nested list. A list may contain numbers or other lists, nested any number of times: `[1, [2, [3, 4], 5], 6]`. To total every number, handle the two cases:

```
def deep_sum(items):
    total = 0
    for x in items:
        if isinstance(x, list):   # a sub-structure:
            total += deep_sum(x)  #   recurse into it
        else:                     # a plain value:
            total += x            #   add it directly
    return total
```

On a plain number the function adds it; on a list it **calls itself** on that list. The nesting in the data is mirrored by nesting in the calls.

Definition 8 (Recursion, base case, call stack) A **recursive function** solves a problem by calling itself on a smaller piece of the same problem. It needs two parts: a **base case** that returns directly without recursing (here, a plain value — and an empty list, which the loop handles by returning `0`), and a **recursive case** that breaks the problem down and calls

itself. The **call stack** is the stack of function calls currently in progress: each recursive call is pushed on top, and pops off when it returns.

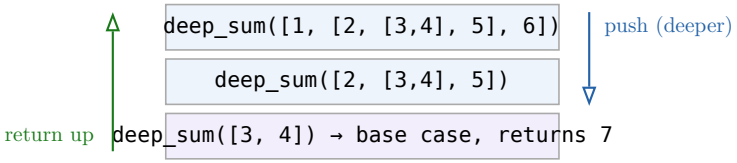

Figure 5: The call stack for `deep_sum` descending into `[1,[2,[3,4],5],6]`. Each deeper list pushes a new frame; reaching plain values (the base case) stops the descent, and results return back up the stack.

4.3 Recursion on real scraped structures

The two structures the OP names are exactly the two you scrape:

Worked example — Walking a nested JSON comment thread. An API returns comments, each with a `replies` list of the same shape:

```
def count_comments(node):
    total = 1                          # count this comment
    for reply in node.get("replies", []):
        total += count_comments(reply) # recurse into each reply
    return total
```

The base case is implicit: a comment with no replies runs the loop zero times and returns `1`. Try to do this with fixed nested `for` loops and you must guess the maximum depth — recursion needs no such guess.

Common pitfall Two recursion failures dominate. **Missing or wrong base case**: if the function always recurses, it never stops — Python halts it with `RecursionError: maximum recursion depth exceeded`. Always ask "on the smallest input, does this return **without** recursing?" **Forgetting to combine results**: writing `count_comments(reply)` without `total +=` recurses correctly but throws the answer away. Each recursive call returns a value you must **use**.

Remark Anything recursive can in principle be rewritten as a loop with an explicit stack or queue, and for very deep structures that avoids Python's recursion limit (about 1000 frames). But for the moderate nesting of real scraped data, the recursive version mirrors the data's shape so closely that it is almost always clearer — and clarity is worth more than cleverness here.

Chapter takeaways. Recursion processes data nested to an unknown depth by having a function call itself on each sub-structure. Every recursive function needs a **base case** that returns without recursing and a **recursive case** that shrinks the problem; the **call stack** holds the in-progress calls. The natural home of recursion in this course is nested JSON and HTML trees, where a flat loop cannot reach the bottom.

5 Web scraping with requests and BeautifulSoup

5.1 From a page meant for eyes to data meant for code

In DAT12-2 you called an **API** — a service that hands back clean JSON, designed for programs. Most of the web has no such API: the data is buried in HTML meant for a human eye and a browser. **Web scraping** is the craft of fetching that HTML and extracting structured data from it. Two libraries do the work: `requests` to fetch the page, and `BeautifulSoup` to parse it.

5.2 Fetching: requests

Worked example — Downloading a page.

```
import requests
resp = requests.get("https://example.com/products")
resp.raise_for_status()      # stop if the request failed
html = resp.text             # the raw HTML, as a string
```

`requests.get` performs the same HTTP GET you used for APIs in DAT12-2 — but the response body is now a page of HTML, not JSON. `raise_for_status()` turns a failed request (a 404 or 500) into a loud exception instead of a silent wrong answer.

5.3 HTML is a tree

Before parsing, understand the shape of what you are parsing. An HTML document is a **tree** of nested **elements** (tags): `<html>` contains `<body>` contains `<div>`s contains `<p>`s and `<a>`s. Each element may have **attributes** (`class="price"`, `href="..."`) and **text**. Scraping is navigating this tree to the elements that hold your data.

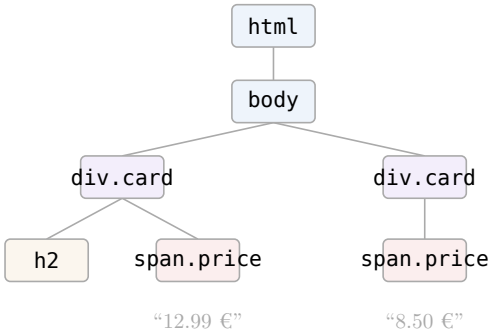

Figure 6: An HTML page is a tree of nested elements. To scrape the prices you navigate to the `span.price` nodes; `find_all("span", class_="price")` returns every such node regardless of where it sits in the tree.

5.4 Parsing: BeautifulSoup

Definition 9 (BeautifulSoup navigation) `BeautifulSoup(html, "html.parser")` parses an HTML string into a navigable tree. The core methods:

- `soup.find(tag, ...)` — the **first** matching element, or `None`.
- `soup.find_all(tag, ...)` — a **list** of all matching elements.
- select by attribute: `find_all("span", class_="price")`, or with CSS selectors via `soup.select(".card .price")`.
- from an element: `.text` (its text content), `["href"]` (an attribute), `.find(...)` (search within it).

Worked example — Extracting a list of products. Given the tree above, pull each product's name and price:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
products = []
for card in soup.find_all("div", class_="card"):
    name = card.find("h2").text.strip()
    price = card.find("span", class_="price").text.strip()
    products.append({"name": name, "price": price})
```

The pattern is universal: `find_all` the **repeating container** (each product card), then `find` the fields **within** each one. Looping over containers and reaching inside keeps each product's fields together.

Common pitfall Scraping breaks in ways API calls do not, because you are reading a layout, not a contract. **Never assume an element exists**: `card.find("h2").text` crashes with `AttributeError: 'NoneType' has no attribute 'text'` the first time a card has no `<h2>`. Guard every extraction (the next chapter shows how). And the HTML can change overnight without warning — the site owner owes your scraper nothing — so scrapers are inherently fragile and need maintenance.

Chapter takeaways. Scraping fetches human-facing HTML (`requests.get`, guarded by `raise_for_status`) and extracts data from it. HTML is a tree of nested, attributed elements; `BeautifulSoup` navigates it with `find` / `find_all` and CSS `select`. The reliable pattern is `find_all` the repeating container, then `find` each field inside it — never assuming an element is present.

6 Handling scraping challenges

6.1 Real sites fight back

The toy example assumed one page that always has every element. Real scraping faces four recurring obstacles: data spread across many pages, servers that object to rapid requests, elements that are sometimes missing, and structures nested several levels deep. Handling these is most of the actual work.

6.2 Pagination

Data rarely fits on one page; a catalogue spreads over `?page=1`, `?page=2`, … **Pagination** means visiting each page in turn and accumulating the results, until there are no more.

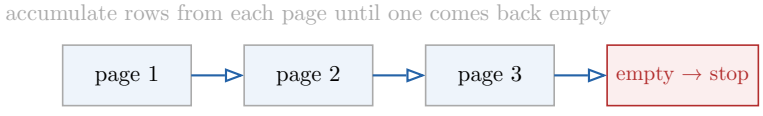

Figure 7: Pagination as a loop: fetch a page, extract its rows, advance to the next, and stop when a page yields nothing (or a "next" link is absent). Never hard-code the page count — let the data decide where to stop.

Worked example — Looping over pages until empty.

```
rows = []
page = 1
while True:
    resp = requests.get(url, params={"page": page})
    items = BeautifulSoup(resp.text, "html.parser").find_all("div", class_="card")
    if not items:            # empty page → we are past the end
        break
    rows.extend(parse(card) for card in items)
    page += 1
```

The loop's **stopping condition is in the data**, not a number you guessed. A site might gain or lose pages tomorrow; this loop adapts.

6.3 Rate limiting and politeness

Definition 10 (Rate limiting) A server **rate-limits** when it rejects or slows requests that arrive too fast, to protect itself — often replying with HTTP status `429 Too Many Requests`. A polite scraper stays under the limit by **pausing between requests** (`time.sleep`), identifying itself with a `User-Agent` header, and **backing off** (waiting longer, then retrying) when it does hit a limit.

Worked example — A pause between requests.

```
import time
for page in range(1, 11):
    fetch(page)
    time.sleep(1)     # one request per second — gentle on the server
```

One second per request feels slow, but it is the difference between a scraper that runs for years and one that gets your IP blocked on day one. **Slow and allowed** beats **fast and banned**.

6.4 Missing elements

The defensive habit the previous chapter warned about: scraped pages are irregular, so **every** extraction must tolerate an absent element rather than crashing the whole run.

Worked example — Guarding an extraction.

```
h2 = card.find("h2")
name = h2.text.strip() if h2 else None   # missing → None, not a crash
```

`find` returns `None` when nothing matches; check for it before reaching for `.text`. Recording `None` for a missing field keeps the run alive and the gap honest — far better than a crash on row 4 873 of 5 000.

6.5 Nested structures

When the data nests deeply — categories inside categories, a comment tree — reach for the recursion of the earlier chapter. Scraping and recursion meet here: `find_all` gets you the repeating containers at one level; recursion descends through the levels when their number is unknown.

Common pitfall The most damaging scraping bug is the run that crashes near the end and loses everything. Three habits prevent it: guard every extraction against missing elements; let pagination stop on the data, not a guessed count; and pace requests so you are never

blocked mid-run. A scraper that collects 90% of the data without dying beats one that aims for 100% and crashes at row 4 999.

Chapter takeaways. Paginate by looping until a page yields nothing — never hard-code the count. Respect rate limits: pause between requests, send a `User-Agent`, and back off on `429`. Guard every extraction so a missing element records `None` instead of crashing. Descend genuinely nested structures with recursion. Aim for a run that survives irregularity, not a brittle one chasing completeness.

7 Storing and documenting scraped data

7.1 Data you collected is data you must be able to trust later

A scrape that lives only in memory is gone the moment the script ends. You must **persist** it in a structured format, and — because you collected it yourself, so no one else can vouch for it — **document how you collected it**. The two formats the OP names cover the two common needs.

Definition 11 (CSV vs SQLite) **CSV** is a plain-text table: one line per row, fields separated by commas. Simple, universal, readable in any tool — but untyped and unindexed, and awkward once you have several related tables. **SQLite** is a single-file SQL database: it stores **typed** columns, enforces structure, handles multiple related tables, and answers queries — at the cost of not being human-readable in a text editor.

Worked example — Saving the same data two ways.

```
import pandas as pd, sqlite3
df = pd.DataFrame(rows)

df.to_csv("products.csv", index=False)        # → a portable text table

conn = sqlite3.connect("products.db")
df.to_sql("products", conn, if_exists="replace", index=False)  # → a queryable table
```

Reach for CSV when the result is a single table you will hand to a colleague or reopen in pandas. Reach for SQLite when you scrape repeatedly, relate several tables, or want to query without loading everything into memory.

7.2 Documenting the collection process

Definition 12 (A data collection record) Because self-collected data has no external provenance, you must record, beside the data itself: the **source** URLs scraped; the **date and time** of collection (web data changes); the **method** (tool, pagination range, any filters applied); **how missing or malformed records were handled**; and **known limitations** (pages that failed, fields often absent). Without this, the dataset cannot be interpreted, reproduced, or defended.

Common pitfall Scraped data is a **snapshot**, not a live feed: prices, listings, and counts reflect the moment you scraped, and re-running tomorrow may give different numbers — not

because of a bug but because the world moved. Always stamp the collection date in the data and the record. An undated scraped dataset is almost impossible to interpret six months on.

Chapter takeaways. Persist scraped data in a structured format — CSV for a single portable table, SQLite for typed, related, queryable tables. Because the data is self-collected, document its provenance: source URLs, collection date/time, method, how gaps were handled, and known limitations. Treat every scrape as a dated snapshot.

8 Legal and ethical boundaries

8.1 Just because you can fetch it does not mean you may

Scraping is powerful, and that power has limits that are not technical but legal and ethical. A competent scraper who ignores them creates real risk — for themselves and their employer. This is not optional background; it is part of doing the work responsibly.

Definition 13 (What to check before scraping)

- **Terms of Service**: many sites' terms explicitly forbid automated access; violating them can carry legal consequences.
- **`robots.txt`**: a file at a site's root stating which paths automated agents are asked not to visit. It is a request, not a lock, but ignoring it is acting in bad faith.
- **Personal data**: scraping data about identifiable people triggers privacy law (e.g. the GDPR in the EU) regardless of whether the page is public.
- **Copyright**: page **content** may be protected; collecting facts differs from republishing someone's text or images.
- **Server load**: hammering a site can disrupt its service — itself potentially unlawful, and certainly unethical.

Worked example — Three scrapes on a spectrum.

- **Clearly fine**: a government open-data page that publishes statistics for reuse, scraped gently, facts only.
- **Clearly not**: harvesting users' names, photos, and locations from a social network whose terms forbid it, to build a profile database — a privacy and terms violation at once.
- **It depends**: prices from a competitor's public shop — often legal as facts, but possibly against their terms; check, throttle, and take only what you need.

Common pitfall "The page is public" is **not** a green light. Public visibility says nothing about whether the terms permit automated collection, whether the data is personal, or whether the content is copyrighted. When in doubt, prefer an official API or published dataset, ask permission, and collect the minimum you need. The right question is never "can I technically get this?" but "am I permitted to, and should I?"

Chapter takeaways. Before scraping, check the site's Terms of Service and `robots.txt`, consider whether the data is personal (privacy law) or copyrighted, and avoid overloading the

server. Public access does not imply permission. When unsure, prefer an official source, ask, and take only what you need.

9 Building a reusable data collection module

9.1 From a throwaway script to a tool

Your first scrape is a script you run once. The tenth time you adapt it, you realise you have been copying and editing the same code — and every copy has its own bugs. The professional move is to package the collection logic into a **reusable module**: functions with clear inputs, outputs, and error handling, so the next dataset is a function call, not a rewrite. This builds directly on the functions and exception handling of DAT12-2.

Definition 14 (A reusable collection module) A **reusable data collection module** exposes a small set of functions, each with: **clear inputs** (typed parameters — the URL, page range, filters); a **clear output** (a predictable return, e.g. a DataFrame or list of dicts); **error handling** (failed requests, missing fields, and rate limits handled inside, so callers get clean data or a clear error, not a crash); and **separation of concerns** — fetching, parsing, and storing as distinct functions that can be tested and reused independently.

Worked example — Three functions with one job each.

```
def fetch_page(url, page):
    """Fetch one page; raise on HTTP error, return HTML text."""
    resp = requests.get(url, params={"page": page}, headers=HEADERS)
    resp.raise_for_status()
    return resp.text

def parse_products(html):
    """Parse HTML → list of product dicts, guarding missing fields."""
    ...

def collect(url, max_pages=None):
    """Paginate, parse, and return one combined DataFrame."""
    ...
```

Each function does one thing and has a docstring stating its contract. `collect` **composes** the other two. Because fetching is separate from parsing, you can test the parser on saved HTML without hitting the network at all.

Common pitfall The anti-pattern is one giant function that fetches, parses, paginates, cleans, and saves in a single 150-line block with a bare `except: pass` swallowing every error. When it breaks — and a scraper **will** — you cannot tell which part failed, and the bare `except` has hidden the reason. Keep the stages separate, let each raise meaningful errors, and catch exceptions **specifically** (a failed request) rather than silencing all of them.

Chapter takeaways. Package collection logic into reusable functions with typed inputs, predictable outputs, and real error handling. Separate fetching, parsing, and storing so each

can be tested and reused independently. Avoid the monolith-with-`except: pass`; handle specific errors and surface the rest.

10 Exploratory analysis on self-collected data

10.1 Closing the loop: from collection to question

The course's two halves meet here. You used advanced pandas and algorithmic thinking to **process** data efficiently; you used scraping to **collect** data that did not exist as a dataset before. Now you turn the pandas skills of Pandas I and this course onto your **own** freshly scraped data — and, crucially, end where every data project must: with **hypotheses worth investigating further**.

Definition 15 (Exploratory data analysis (EDA)) **Exploratory data analysis** is the open-ended first look at a dataset: checking its shape and types, summarising distributions (`describe`), counting categories (`value_counts`), inspecting missingness, and plotting relationships — in order to understand what the data **is** and what questions it might answer, before any formal modelling.

Worked example — From a scraped table to three hypotheses. You scraped 2 000 second-hand car listings (price, year, mileage, brand). Exploring:

- `describe()` shows prices from €500 to €90 000 — a few luxury outliers to watch.
- `groupby("brand")["price"].mean()` ranks brands by average price.
- a scatter of price vs mileage slopes downward.

These observations become **testable hypotheses**: (1) price falls roughly linearly with mileage; (2) German brands command a premium at equal mileage; (3) listings under €1 000 are disproportionately missing a service history. Each is a sentence you could now go and test — which is exactly what EDA is for.

Common pitfall Self-collected data carries **collection bias** that EDA must surface, not hide. If you scraped only the first 10 pages, sorted by "most popular", your sample is not the whole market — it over-represents popular items. State such limitations beside every finding: a hypothesis drawn from a biased sample is a hypothesis about **that sample**, not the world, until confirmed on representative data.

Chapter takeaways. Turn your scraped dataset into understanding with exploratory analysis — shape, types, distributions, counts, missingness, relationships — then end with at least three concrete, testable hypotheses. Keep the data's collection bias in view: findings are about your sample until shown to generalise.

Exercises

1. Take a sales table with `Country` and `Year` columns. Build a MultiIndex on both, then (a) select one country's block, (b) take the 2026 cross-section across all countries with `xs`, and (c) `unstack` the year level into columns. Explain what each result answers.

2. Given a column of noisy daily values, compute a 7-day **rolling** mean and an **expanding** mean. Plot all three series and explain, in words, the difference between what the two windows show — and why a trailing window matters if the result feeds a forecast.
3. Load a CSV of dated readings. Convert the date column with `pd.to_datetime`, set it as the index, and `resample` to a monthly mean. Then deliberately leave the dates as strings and show one operation that silently breaks.
4. Write two functions that detect duplicates in a list of IDs — one $O(n^2)$ (compare all pairs) and one $O(n)$ (use a set). State the Big O of each, then time both on 1 000 and 100 000 IDs and report how the gap grows.
5. Profile a small pipeline (load, clean, a row loop, save) with `time.perf_counter` around each step. Identify the bottleneck, replace it with a vectorized operation, and report the before/after timings — confirming the result is unchanged.
6. Write a recursive function that totals (or counts the leaves of) an arbitrarily nested list or JSON structure. Identify its base case explicitly, and demonstrate what happens if you remove the base case.
7. Using `requests` and `BeautifulSoup`, scrape the items from a single permitted page into a list of dicts. Apply the "`find_all` the container, `find` each field" pattern and guard every field against being missing.
8. Extend the scraper to paginate until a page returns no items, adding a `time.sleep` between requests and a `User-Agent` header. Explain how your loop decides to stop and why you did not hard-code the page count.
9. Save your scraped data to both a CSV and a SQLite database. Write a short collection record (source URLs, date/time, method, gap handling, limitations) and store it alongside the data.
10. For a site you intend to scrape, locate its `robots.txt` and Terms of Service, note whether the data is personal or copyrighted, and write a paragraph judging whether scraping it is permissible and why.
11. Refactor your scraping code into a reusable module with separate `fetch` / `parse` / `collect` functions, each with a docstring, typed inputs, a predictable output, and specific error handling. Test the parser on saved HTML without making a network request.
12. Run an exploratory analysis on your own scraped dataset (shape, types, `describe`, `value_counts`, one plot). Conclude with at least three concrete, testable hypotheses, and state one collection bias that limits them.